

KIỂM THỬ PHẦN MỀM

ĐỀ TÀI :

Ứng dụng Đăng nhập & Quản lý Sản phẩm

(Login & Product Management - Version 1.0)

GIẢNG VIÊN: Từ Lăng Phiêu

NHÓM: 04

SINH VIÊN:

Kiều Hoài Nam - 3123410226

Lê Quốc Cường - 3123410040

Lê Công Vinh - 3123410431

Phan Vinh Thái - 3123560079

Nguyễn Dương Khang - 3123410151

Mục lục

Đóng Góp Của Nhóm	3
1 GIỚI THIỆU DỰ ÁN	3
1.1 Tổng quan	3
1.2 Công nghệ sử dụng	3
2 CÂU 1: PHÂN TÍCH VÀ THIẾT KẾ TEST CASES	3
2.1 Câu 1.1: Login - Phân tích và Test Scenarios (10 điểm)	3
2.1.1 Yêu cầu (5 điểm)	3
2.1.2 Test Scenarios cho Login (2 điểm)	4
2.1.3 Thiết kế Test Cases chi tiết (5 điểm)	5
2.2 Câu 1.2: Product - Phân tích và Test Scenarios (10 điểm)	7
2.2.1 Yêu cầu (5 điểm)	7
2.2.2 Test Scenarios cho Product (2 điểm)	8
2.2.3 Thiết kế Test Cases chi tiết (5 điểm)	8
3 CÂU 2: UNIT TESTING VÀ TDD	9
3.1 Frontend Unit Tests (TDD Approach)	9
3.1.1 Phát triển unit tests cho validation module của Login	9
3.1.2 Phát triển unit tests cho validation module của Product	13
3.2 Backend Unit Tests (JUnit 5)	15
3.2.1 Login Service Tests	15
3.2.2 Product Service Tests	16
4 CÂU 3: INTEGRATION TESTING	17
4.1 Login - Integration Testing	17
4.1.1 Test tích hợp Login component với API service	17
4.1.2 Test API endpoints của Login với MockMvc	19
4.2 Product - Integration Testing	20
4.2.1 Test Product component với API service	20
4.2.2 Test các API endpoints của Product	21
5 CÂU 4: MOCK TESTING	22
5.1 Login - Mock Testing	22
5.1.1 Frontend Mocking	22
5.1.2 Backend Mocking	23
5.2 Product - Mock Testing	23
5.2.1 Frontend Mocking	23
5.2.2 Backend ProductRepository Mocking	27
6 CÂU 5: AUTOMATION TESTING VÀ CI/CD	27
6.1 E2E Testing với Cypress	27
6.2 CI/CD với GitHub Actions	28
7 PHẦN MỞ RỘNG (BONUS)	28
7.1 Performance Testing	28
7.1.1 k6 Script cho Login API	28
7.2 Security Testing	28

8 KẾT LUẬN	29
8.1 Tài liệu tham khảo	29

Đóng Góp Của Nhóm

Thành viên	Chức vụ	Phần trăm	Đóng góp
Lê Quốc Cường	Trưởng nhóm		
Kiều Hoài Nam	Thành viên		
Lê Công Vinh	Thành viên		
Phan Vĩnh Thái	Thành viên		
Nguyễn Dương Khang	Thành viên		

1 GIỚI THIỆU DỰ ÁN

1.1 Tổng quan

Dự án **FloginFE_BE** là một ứng dụng web full-stack được phát triển để thực hành các kỹ thuật kiểm thử phần mềm. Hệ thống bao gồm các chức năng chính:

- **Login:** Hệ thống đăng nhập bảo mật với validation chặt chẽ.
- **Product Management:** Quản lý sản phẩm với đầy đủ các thao tác CRUD (Create, Read, Update, Delete).

1.2 Công nghệ sử dụng

- **Frontend:** React 18+, Jest, React Testing Library.
- **Backend:** Spring Boot 3.2+, JUnit 5, Mockito.
- **Database:** MySQL.
- **CI/CD:** GitHub Actions.

2 CÂU 1: PHÂN TÍCH VÀ THIẾT KẾ TEST CASES

2.1 Câu 1.1: Login - Phân tích và Test Scenarios (10 điểm)

2.1.1 Yêu cầu (5 điểm)

a) Phân tích yêu cầu chức năng Login (2 điểm)

1. Validation rules cho Username

- Bắt buộc nhập (required, không được để trống).
- Độ dài hợp lệ: 3–50 ký tự.
- Chỉ cho phép ký tự: a–z, A–Z, 0–9, dấu gạch ngang (-), dấu chấm (.), dấu gạch dưới (_).
- Không được chứa khoảng trắng đầu/cuối.
- Không được chứa ký tự đặc biệt ngoài danh sách cho phép.

2. Validation rules cho Password

- Bắt buộc nhập.

- Độ dài hợp lệ: 6–100 ký tự.
- Phải chứa ít nhất 1 chữ cái (a–z hoặc A–Z).
- Phải chứa ít nhất 1 chữ số (0–9).
- Không giới hạn ký tự đặc biệt (nếu hệ thống không cấm).
- Không có khoảng trắng đầu/cuối (trim).

3. Authentication Flow (Luồng xác thực)

1. User nhập username + password.
2. Hệ thống validate input: Nếu invalid → trả về lỗi Validation và không gọi API backend.
3. Nếu input hợp lệ → gửi request đến backend: POST /login { username, password }.
4. Backend kiểm tra: Có tồn tại username không? Password có khớp không?
5. Nếu hợp lệ: Trả về token (JWT), Frontend lưu token, Chuyển hướng người dùng sang products.
6. Nếu không hợp lệ: Trả về lỗi: Tên đăng nhập hoặc mật khẩu không đúng.

4. Error Handling

- Username rỗng → "Username không được để trống".
- Password rỗng → "Password không được để trống".
- Username độ dài nhỏ hơn 3 hoặc lớn hơn 50 → "Username phải từ 3-50 ký tự".
- Password độ dài nhỏ hơn 6 hoặc lớn hơn 100 → "Password phải từ 6-100 ký tự".
- Username sai format → "Username chỉ được chứa chữ cái, số...".
- Password sai format → "Password phải chứa ít nhất 1 chữ cái và 1 chữ số".
- Username không tồn tại → "User không tồn tại!".
- Sai mật khẩu hoặc username → "Tên đăng nhập hoặc mật khẩu không đúng!".

2.1.2 Test Scenarios cho Login (2 điểm)

STT	Test Scenario	Loại test
1	Đăng nhập thành công với username + password hợp lệ	Happy Path
2	Username rỗng	Negative
3	Password rỗng	Negative
4	Username < 3 ký tự	Boundary – Invalid
5	Username > 50 ký tự	Boundary – Invalid
6	Password < 6 ký tự	Boundary – Invalid
7	Password > 100 ký tự	Boundary – Invalid
8	Username chứa ký tự không hợp lệ	Edge case
9	Password không chứa số	Negative (Format)

10	Password không chứa chữ cái	Negative (Format)
11	Sai username	Negative
12	Sai password	Negative
13	Username có khoảng trắng đầu/cuối	Edge case
14	Password có khoảng trắng đầu/cuối	Edge case
15	Server trả lỗi 500	Error handling

Phân loại ưu tiên test scenarios (1 điểm)

- **Critical**

- Đăng nhập thành công (TC1) → Chức năng chính, ảnh hưởng toàn bộ hệ thống.
- Sai username hoặc password (TC11, TC12) → Ảnh hưởng trực tiếp đến bảo mật.
- Username/password rỗng (TC2, TC3) → Validation cơ bản bắt buộc hoạt động.

- **High**

- Password không chứa số/chữ (TC9, TC10) → Vi phạm quy định bảo mật.
- Username chứa ký tự không hợp lệ (TC8).

- **Medium**

- Boundary tests (TC4, TC5, TC6, TC7) → Đảm bảo tính ổn định input.

- **Low**

- Khoảng trắng đầu cuối (TC13, TC14).
- Server 500 (TC15) → Không xảy ra thường xuyên nhưng cần kiểm tra.

2.1.3 Thiết kế Test Cases chi tiết (5 điểm)

Test Case ID	TC_LOGIN_001
Test Name	Đăng nhập thành công với credentials hợp lệ
Priority	Critical
Preconditions	- User account đã tồn tại - Application đang chạy
Test Steps	1. Navigate to login page 2. Enter valid username 3. Enter valid password 4. Click Login button
Test Data	Username: testuser Password: Test123
Expected Result	- Success message displayed - Token stored - Redirect to dashboard
Status	Not Run

Test Case ID	TC_LOGIN_002
Test Name	Đăng nhập thất bại với password sai

Priority	High
Preconditions	- User account đã tồn tại - Application đang chạy
Test Steps	1. Navigate to login page 2. Enter valid username 3. Enter invalid password 4. Click Login button
Test Data	Username: testuser Password: WrongPass123
Expected Result	- Error message hiển thị: "Tên đăng nhập hoặc mật khẩu không đúng!" - Không lưu token - Vẫn ở trang login
Status	Not Run

Test Case ID	TC_LOGIN_003
Test Name	Đăng nhập thất bại với username không tồn tại
Priority	High
Test Steps	1. Navigate to login page 2. Enter invalid username 3. Enter any password 4. Click Login button
Test Data	Username: nonexistentuser Password: AnyPassword123
Expected Result	- Error message hiển thị: "Đăng nhập thất bại!" - Không lưu token - Vẫn ở trang login
Status	Not Run

Test Case ID	TC_LOGIN_004
Test Name	Đăng nhập thất bại khi để trống username/password
Priority	High
Test Steps	1. Navigate to login page 2. Để trống username field 3. Để trống password field 4. Click Login button
Expected Result	- Validation message hiển thị: "Username không được để trống" và "Password không được để trống"
Status	Not Run

Test Case ID	TC_LOGIN_005
Test Name	Đăng nhập thất bại với password không đủ độ dài
Priority	Medium

Test Steps	1. Navigate to login page 2. Enter valid username 3. Enter short password 4. Click Login button
Test Data	Username: testuser Password: pass
Expected Result	- Validation message hiển thị: "Password phải từ 6-100 ký tự" - Không gửi request đến server
Status	Not Run

2.2 Câu 1.2: Product - Phân tích và Test Scenarios (10 điểm)

2.2.1 Yêu cầu (5 điểm)

Phân tích yêu cầu chức năng Product CRUD (2 điểm)

1. Create – Thêm sản phẩm mới

- name: không rỗng, tối đa 200 ký tự.
- description: tối đa 1000 ký tự.
- price: bắt buộc, > 0.
- quantity: >= 0.
- category: tối đa 50 ký tự, phải thuộc danh sách hợp lệ.
- imageUrl: tối đa 500 ký tự.
- isAvailable: true/false.

2. Read – Xem danh sách / chi tiết sản phẩm

- Xem danh sách toàn bộ sản phẩm (phân trang).
- Cho phép tìm kiếm theo tên, category.
- Cho phép xem chi tiết theo ID.

3. Update – Cập nhật thông tin

- Cho phép cập nhật tất cả trừ createdBy.
- Validation giống Create.
- Không cho update nếu sản phẩm không tồn tại.

4. Delete – Xóa sản phẩm

- Xóa bằng ID. Nếu không tồn tại trả về "Product not found".

2.2.2 Test Scenarios cho Product (2 điểm)

STT	Test Scenario	Loại	Kết quả mong đợi
1	Create product thành công	Happy path	Trả về ProductResponse đúng dữ liệu
2	Update product thành công	Happy path	Dữ liệu được cập nhật
3	Delete product thành công	Happy path	Sản phẩm bị xóa
4	Read product by ID thành công	Happy path	Trả về thông tin sản phẩm
5	Create thất bại (Name rỗng)	Negative	Lỗi "Tên sản phẩm không được để trống"
6	Create thất bại (Price <= 0)	Negative	Lỗi "Giá phải lớn hơn 0"
7	Description > 1000 ký tự	Boundary	Lỗi validation
8	Update sản phẩm không tồn tại	Edge case	Lỗi "Không tìm thấy sản phẩm..."
9	Delete ID không tồn tại	Edge case	Lỗi "Không tìm thấy sản phẩm..."
10	Search product theo tên	Functional	Trả về danh sách phù hợp

2.2.3 Thiết kế Test Cases chi tiết (5 điểm)

Test Case ID	TC_PRODUCT_001
Test Name	Tạo sản phẩm mới thành công
Priority	Critical
Preconditions	User đã đăng nhập, có quyền tạo sản phẩm
Test Steps	1. Navigate to Product page 2. Click "Add New Product" 3. Nhập thông tin hợp lệ 4. Click Save
Test Data	Name: Laptop Dell XPS, Price: 25000000, Quantity: 10, Category: Electronics, isAvailable: true
Expected Result	- Sản phẩm tạo thành công - API trả về ProductResponse có ID mới - Hiện thị thông báo thành công
Status	Not Run

Test Case ID	TC_PRODUCT_002
Test Name	Cập nhật sản phẩm thành công
Priority	High
Test Steps	1. Navigate to Product page 2. Chọn sản phẩm ID = 5 3. Click "Edit Product" 4. Nhập thông tin hợp lệ 5. Click Save
Expected Result	- Cập nhật thành công - API trả về ProductResponse mới nhất

Test Case ID	TC_PRODUCT_003
Test Name	Xóa sản phẩm thành công
Priority	Critical

Test Steps	1. Navigate to Product page 2. Chọn sản phẩm ID = 10 3. Click "Delete Product" 4. Confirm Delete
Expected Result	- Xóa thành công - Sản phẩm biến mất khỏi danh sách

Test Case ID	TC_PRODUCT_004
Test Name	Xem chi tiết sản phẩm theo ID
Priority	Medium
Test Steps	1. Navigate to Product page 2. Click vào sản phẩm ID = 3
Expected Result	- API trả về ProductResponse đầy đủ các trường

Test Case ID	TC_PRODUCT_005
Test Name	Tạo sản phẩm thất bại do giá không hợp lệ
Priority	High
Test Steps	1. Navigate to Product page 2. Click "Add New Product" 3. Nhập Price = 0 4. Click Save
Expected Result	- Hệ thống trả lỗi validation: "Giá phải lớn hơn 0" - API trả HTTP 400

3 CÂU 2: UNIT TESTING VÀ TDD

Nhóm áp dụng quy trình TDD (Test-Driven Development) theo chu trình: **Red** (Viết test fail) → **Green** (Viết code để pass test) → **Refactor** (Tối ưu code).

3.1 Frontend Unit Tests (TDD Approach)

3.1.1 Phát triển unit tests cho validation module của Login

Bước 1: Red (Viết Test)

```

1  import { validateUsername, validatePassword } from '../utils/validation';
2  describe('Validation Utils Tests', () => {
3    // ===== a) Unit tests cho validateUsername() (2 điểm)
4    // =====
5    describe('validateUsername()', () => {
6      // Test username rỗng
7      test('Nên trả về lỗi khi username rỗng', () => {
8        expect(validateUsername('')).toBe('Username không được để trống');
9        expect(validateUsername(null)).toBe('Username không được để trống');
10       expect(validateUsername(undefined)).toBe('Username không được để trống');
11     });
12     // Test username chỉ có khoảng trắng
13     test('Nên trả về lỗi khi username chỉ có khoảng trắng', () => {
14       expect(validateUsername('   ')).toBe('Username không được để trống');
15     });
16   });
17 }

```

```

14     expect(validateUsername('')).toBe('Username không được để trống');
15   });
16   // Test username quá ngắn
17   test('Nên trả về lỗi khi username quá ngắn (< 3 ký tự)', () => {
18     expect(validateUsername('a')).toBe('Username phải có ít nhất 3 ký tự');
19     expect(validateUsername('ab')).toBe('Username phải có ít nhất 3 ký t
20   });
21   // Test username quá dài
22   test('Nên trả về lỗi khi username quá dài (> 20 ký tự)', () => {
23     const longUsername = 'a'.repeat(21);
24     expect(validateUsername(longUsername)).toBe('Username không được vuo
25     expect(validateUsername('abcdefghijklmnopqrstu')).toBe('Username khô
26   });
27   // Test ký tự đặc biệt không hợp lệ
28   test('Nên trả về lỗi khi username chứa ký tự đặc biệt không hợp lệ',
29   () => {
30     expect(validateUsername('user@name')).toBe('Username chỉ được chứa
31     expect(validateUsername('user name')).toBe('Username chỉ được chứa
32     expect(validateUsername('user-name')).toBe('Username chỉ được chứa
33     expect(validateUsername('user#123')).toBe('Username chỉ được chứa ch
34     expect(validateUsername('user.name')).toBe('Username chỉ được chứa
35   });
36   // Test username hợp lệ
37   test('Nên trả về null khi username hợp lệ', () => {
38     expect(validateUsername('abc')).toBeNull();
39     expect(validateUsername('user123')).toBeNull();
40     expect(validateUsername('test_user')).toBeNull();
41     expect(validateUsername('User_123')).toBeNull();
42     expect(validateUsername('abcdefghijklmnpqrst')).toBeNull();
43   });
44   // Test edge cases
45   test('Nên xử lý đúng các edge cases', () => {
46     expect(validateUsername('___')).toBeNull(); // Chỉ gạch dưới
47     expect(validateUsername('123')).toBeNull(); // Chỉ số
48     expect(validateUsername('ABC')).toBeNull(); // Chỉ hoa
49     expect(validateUsername('abc123___')).toBeNull(); // Mix hợp lệ
50   });
51 });
52 // ===== b) Unit tests cho validatePassword() (2 điểm)
53 // =====
54 describe('validatePassword()', () => {
55   // Test password rỗng
56   test('Nên trả về lỗi khi password rỗng', () => {
57     expect(validatePassword('')).toBe('Password không được để trống');
58     expect(validatePassword(null)).toBe('Password không được để trống');
59     expect(validatePassword(undefined)).toBe('Password không được để tró
60   });
61   // Test password chỉ có khoảng trắng
62   test('Nên trả về lỗi khi password chỉ có khoảng trắng', () => {

```

```

62     expect(validatePassword('    ')).toBe('Password không được để trống');
63     expect(validatePassword('  ')).toBe('Password không được để trống');
64   });
65   // Test password quá ngắn
66   test('Nên trả về lỗi khi password quá ngắn (< 6 ký tự)', () => {
67     expect(validatePassword('12345')).toBe('Password phải có ít nhất 6 ký
68     tự');
69     expect(validatePassword('abc')).toBe('Password phải có ít nhất 6 ký
70     tự');
71     expect(validatePassword('a')).toBe('Password phải có ít nhất 6 ký tự');
72     expect(validatePassword('Test1')).toBe('Password phải có ít nhất 6 ký
73     tự');
74   });
75   // Test password quá dài
76   test('Nên trả về lỗi khi password quá dài (> 30 ký tự)', () => {
77     const longPassword = 'A1' + 'a'.repeat(30);
78     expect(validatePassword(longPassword)).toBe('Password không được vượt
79     quá 30 ký tự');
80     expect(validatePassword('Test123' + 'a'.repeat(25))).toBe('Password
81     không được vượt quá 30 ký tự');
82   });
83   // Test password không có chữ cái
84   test('Nên trả về lỗi khi password không có chữ cái', () => {
85     expect(validatePassword('123456')).toBe('Password phải chứa ít nhất
86     một chữ cái');
87     expect(validatePassword('!@#$$%^')).toBe('Password phải chứa ít nhất
88     một chữ cái');
89     expect(validatePassword('12345678')).toBe('Password phải chứa ít nhất
90     một chữ cái');
91   });
92   // Test password không có số
93   test('Nên trả về lỗi khi password không có số', () => {
94     expect(validatePassword('abcdef')).toBe('Password phải chứa ít nhất
95     một chữ số');
96     expect(validatePassword('Password')).toBe('Password phải chứa ít nhất
97     một chữ số');
98     expect(validatePassword('TestPass')).toBe('Password phải chứa ít nhất
99     một chữ số');
100   });
101   // Test password hợp lệ
102   test('Nên trả về null khi password hợp lệ', () => {
103     expect(validatePassword('Pass123')).toBeNull();
104     expect(validatePassword('Test123')).toBeNull();
105     expect(validatePassword('abcdef1')).toBeNull();
106     expect(validatePassword('MyP4ssw0rd')).toBeNull();
107     expect(validatePassword('Test@123')).toBeNull(); // Có ký tự đặc biệt
108     // vẫn OK
109     expect(validatePassword('a1b2c3')).toBeNull(); // Đúng 6 ký tự (min)
110     expect(validatePassword('Test123' + 'a'.repeat(23))).toBeNull(); //
111     30 ký tự (max)
112   });
113   // Test edge cases - password với ký tự đặc biệt
114   test('Nên chấp nhận password có ký tự đặc biệt (nếu có chữ và số)', ()
115   => {
116     expect(validatePassword('P@ssw0rd')).toBeNull();
117     expect(validatePassword('Test!123')).toBeNull();
118     expect(validatePassword('My#Pass1')).toBeNull();
119   });

```

```

106 // Test edge cases - mix chữ hoa/thường/số
107 test('Nên chấp nhận password mix chữ hoa, thường và số', () => {
108   expect(validatePassword('ABCdef123')).toBeNull();
109   expect(validatePassword('Test123TEST')).toBeNull();
110   expect(validatePassword('aB1cD2eF3')).toBeNull();
111 });
112 });
113 // ===== c) Coverage tests (1 điểm) =====
114 describe('Edge Cases và Coverage', () => {
115   test('validateUsername - test với giá trị boolean/number', () => {
116     // Test với các kiểu dữ liệu khác
117     expect(validateUsername(123)).toBe('Username không được để trống');
118     expect(validateUsername(true)).toBe('Username không được để trống');
119     expect(validateUsername(false)).toBe('Username không được để trống');
120   });
121   test('validatePassword - test với giá trị boolean/number', () => {
122     // Test với các kiểu dữ liệu khác
123     expect(validatePassword(123456)).toBe('Password không được để trống');
124     expect(validatePassword(true)).toBe('Password không được để trống');
125     expect(validatePassword(false)).toBe('Password không được để trống');
126   });
127   test('validateUsername - test boundary values', () => {
128     // Boundary: đúng 3 ký tự (min valid)
129     expect(validateUsername('abc')).toBeNull();
130     // Boundary: đúng 20 ký tự (max valid)
131     expect(validateUsername('12345678901234567890')).toBeNull();
132     // Boundary: 2 ký tự (min - 1)
133     expect(validateUsername('ab')).toBe('Username phải có ít nhất 3 ký tự');
134     // Boundary: 21 ký tự (max + 1)
135     expect(validateUsername('123456789012345678901')).toBe('Username không được vượt quá 20 ký tự');
136   });
137   test('validatePassword - test boundary values', () => {
138     // Boundary: đúng 6 ký tự (min valid)
139     expect(validatePassword('Pass12')).toBeNull();
140     // Boundary: đúng 30 ký tự (max valid)
141     expect(validatePassword('Pass1' + 'a'.repeat(25))).toBeNull();
142     // Boundary: 5 ký tự (min - 1)
143     expect(validatePassword('Pas12')).toBe('Password phải có ít nhất 6 ký tự');
144     // Boundary: 31 ký tự (max + 1)
145     expect(validatePassword('Pass1' + 'a'.repeat(26))).toBe('Password không được vượt quá 30 ký tự');
146   });
147 });
148 });

```

Listing 1: Login Validation Test Script (Red Phase)

Bước 2: Green (Viết Code)

```

1 if (!name || name.trim() === '') { return 'Tên sản phẩm không được để trống'; }
2 // Yêu cầu: 3-100 ký tự
3 if (name.length < 3 || name.length > 100) { return 'Tên sản phẩm phải từ 3-100 ký tự'; }
4 return ''; };

```

```

5
6 export const validatePrice = (price) => {
7   if (price === null || price === undefined || price === '') { return 'Giá
   không được để trống'; }
8   const numPrice = Number(price);
9   if (isNaN(numPrice)) {return 'Giá phải là số'; }
10  if (numPrice <= 0) {return 'Giá phải lớn hơn 0'; }
11  // Yêu cầu: Max 999,999,999
12  if (numPrice > 999999999) { return 'Giá quá lớn (tối đa 999,999,999)'; }
13  return ''; };
14
15 export const validateQuantity = (quantity) => {
16   if (quantity === null || quantity === undefined || quantity === '') {
   return 'Số lượng không được để trống'; }
17   const numQuantity = Number(quantity);
18   if (isNaN(numQuantity)) { return 'Số lượng phải là số'; }
19   if (!Number.isInteger(numQuantity)) { return 'Số lượng phải là số nguyên
   '; }
20   if (numQuantity < 0) { return 'Số lượng không được âm'; }
21   if (numQuantity > 99999) {return 'Số lượng không được quá 99,999'; }
22   return ''; };
23
24 export const validateDescription = (description) => {
25   if (description && description.length > 500) {return 'Mô tả không được
   quá 500 ký tự'; }
26   return '';};
27 export const validateCategory = (category) => {
28   const VALID_CATEGORIES = ['Electronics', 'Fashion', 'Food', 'Books'];
29   if (!category || category.trim() === '') { return 'Danh mục không đ
   ược để trống'; }
30   if (!VALID_CATEGORIES.includes(category)) { return 'Danh mục không hợp
   lệ'; }
31   return ''; };
32 /**
33  * Format currency (VND)
34  */
35 export const formatCurrency = (amount) => {
36   return new Intl.NumberFormat('vi-VN', {
37     style: 'currency',
38     currency: 'VND'
39   }).format(amount);
40 };
41 /**
42  * Format date
43  */
44 export const formatDate = (dateString) => {
45   const date = new Date(dateString);
46   return date.toLocaleDateString('vi-VN');
47 };

```

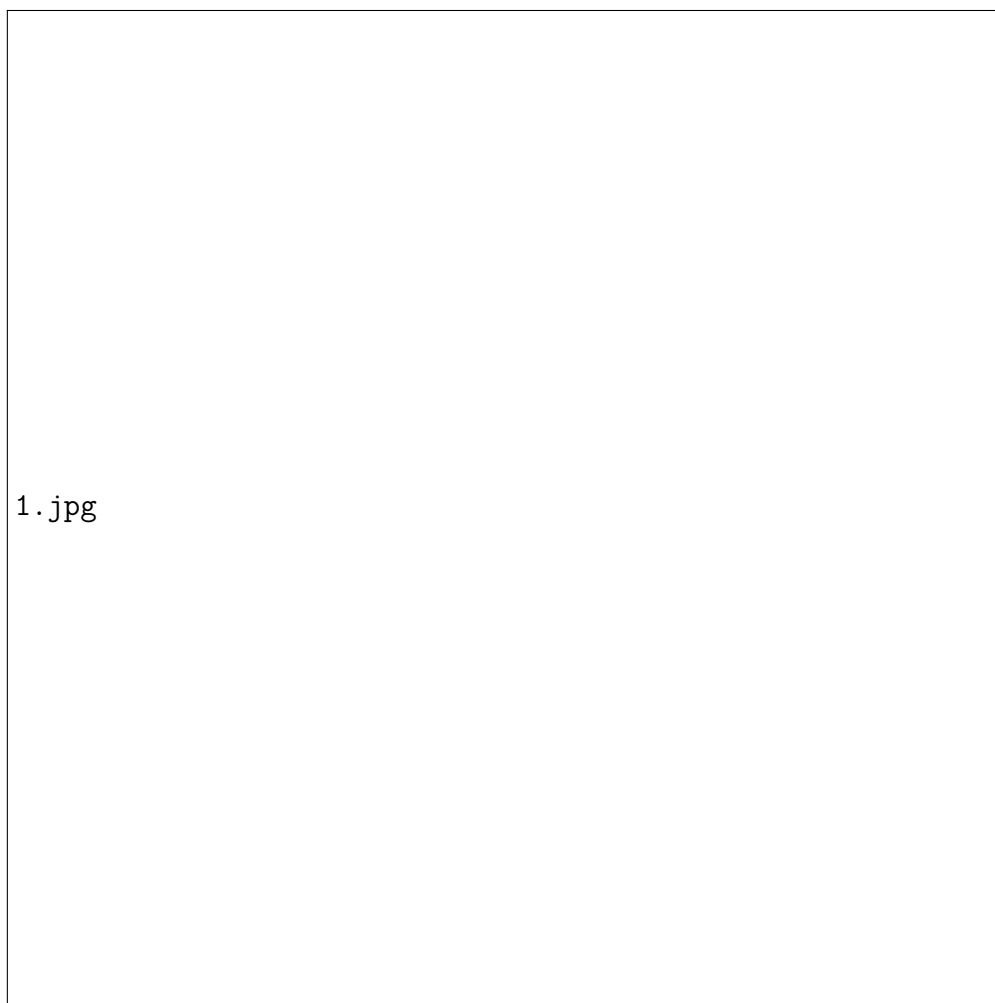
Listing 2: TDD Cycle 1 - Implementation (Green)

Bước 3:Refactor (Tối ưu) Do logic validation product khá nhiều điều kiện if-else, trong bước Refactor, nhóm tiến hành kiểm tra lại coverage để đảm bảo các nhánh (branch) điều kiện như giá min/max, số lượng min/max đều được bao phủ.

3.1.2 Phát triển unit tests cho validation module của Product

Tương tự Login, module Product cũng phát triển theo TDD.

Bước 1: Red (Viết Test Case)



Hình 1: Mô tả ảnh

```

1 import { validateProductName, validatePrice, validateQuantity } from '../
  utils/validation.js';
2
3 describe('Product Validation Utilities Tests', () => {
4   describe('validateProductName', () => {
5     test('returns error for empty product name', () => {
6       expect(validateProductName('')).toBe('Tên sản phẩm không được
  để trống');
7     });
8     test('returns error for short product name (< 3 chars)', () => {
9       expect(validateProductName('AB')).toBe('Tên sản phẩm phải từ
  3-100 ký tự');
10    });
11  });
12
13  describe('validatePrice', () => {
14    test('returns error for empty price', () => {
15      expect(validatePrice('')).toBe('Giá không được để trống');
16    });
17    test('returns error for price <= 0', () => {
18      expect(validatePrice(0)).toBe('Giá phải lớn hơn 0');
19    });
20  });
21
22  // ... validateQuantity tests ...
23 });

```

Listing 3: Product Validation Test Script

Bước 2: Green (Viết Code Logic)

```

1 export const validateProductName = (name) => {
2   if (!name || name.trim() === '') {
3     return 'Tên sản phẩm không được để trống';
4   }
5   if (name.length < 3 || name.length > 100) {
6     return 'Tên sản phẩm phải từ 3-100 ký tự';
7   }
8   return '';
9 };
10
11 export const validatePrice = (price) => {
12   if (price === null || price === undefined || price === '') {
13     return 'Giá không được để trống';
14   }
15   const numPrice = Number(price);
16   if (isNaN(numPrice)) { return 'Giá phải là số'; }
17   if (numPrice <= 0) { return 'Giá phải lớn hơn 0'; }
18   return '';
19 };

```

Listing 4: Product Implementation

3.2 Backend Unit Tests (JUnit 5)

3.2.1 Login Service Tests

```

1 @ExtendWith(MockitoExtension.class)
2 @DisplayName("TC1: Login thành công với credentials hợp lệ")

```



```

3 public class AuthServiceTest {
4     @Mock private UserRepository userRepository;
5     @Mock private PasswordEncoder passwordEncoder;
6     @Mock private AuthenticationManager authenticationManager;
7     @Mock private JwtUtils jwtUtils;
8     @Mock private Authentication authentication;
9     @InjectMocks private AuthService authService;
10
11     // ... setup vars ...
12
13     @Test
14     void testLogin_Success() {
15         when(authenticationManager.authenticate(any(
16 UsernamePasswordAuthenticationToken.class)))
17             .thenReturn(authentication);
18         when(userRepository.findByUsername(anyString())).thenReturn(
19 Optional.of(user));
20
21         AuthResponse response = authService.login(loginRequest);
22         assertNotNull(response);
23         assertEquals("fake-jwt-token", response.getToken());
24         verify(jwtUtils).generateToken("testuser");
25     }
26
27     @Test
28     @DisplayName("TC2: Login that bai voi username khong ton tai")
29     void testLogin_UserNotFound() {
30         when(userRepository.findByUsername(anyString())).thenReturn(
31 Optional.empty());
32         RuntimeException exception = assertThrows(RuntimeException.class,
33 () -> {
34             authService.login(loginRequest);
35         });
36         assertEquals("User không tồn tại!", exception.getMessage());
37     }
38 }

```

Listing 5: AuthService Test

3.2.2 Product Service Tests

```

1 @DisplayName("Product Service Unit tests")
2 public class ProductServiceTest {
3     private ProductService productService;
4     private ProductRepository productRepository; // Mocked manually in
5     // ... setup fake lists ...
6
7     @Test
8     @DisplayName("TC1: Tao san pham moi thanh cong")
9     void testCreateProduct() {
10         ProductRequestDTO request = new ProductRequestDTO("Asus",
11 BigDecimal.valueOf(15000000), 10, "Desc", 1L);
12         ProductResponseDTO result = productService.createProduct(request);
13         assertNotNull(result);
14         assertEquals("Asus", result.getProductName());
15     }
16
17     // ... other tests (update, delete, get) ...

```

Listing 6: ProductService Test with Fake Repository

4 CÂU 3: INTEGRATION TESTING

4.1 Login - Integration Testing

4.1.1 Test tích hợp Login component với API service

a) Test rendering & user interactions

```

1 test('Rendering component và user interactions cơ bản', () => {
2   render(
3     <BrowserRouter future={{ v7_startTransition: true,
4       v7_relativeSplatPath: true }}>
5       <Login />
6     </BrowserRouter>
7   );
8   // Kiểm tra rendering đầy đủ - dùng getByRole để tránh duplicate
9   expect(screen.getByTestId('username-input')).toBeInTheDocument();
10  expect(screen.getByTestId('password-input')).toBeInTheDocument();
11  expect(screen.getByTestId('login-button')).toBeInTheDocument();
12  expect(screen.getByRole('heading', { name: /Đăng Nhập/i })).
13    toBeInTheDocument(); // Chỉ lấy h2
14  expect(screen.getByText(/Chào mừng bạn trở lại/i)).toBeInTheDocument();
15  // Test user interactions - username
16  const usernameInput = screen.getByTestId('username-input');
17  fireEvent.change(usernameInput, { target: { value: 'testuser' } });
18  expect(usernameInput.value).toBe('testuser');
19  // Test user interactions - password
20  const passwordInput = screen.getByTestId('password-input');
21  fireEvent.change(passwordInput, { target: { value: 'Test123' } });
22  expect(passwordInput.value).toBe('Test123');
23 }

```

Listing 7: Rendering test

b) Test form submission API calls

```

1 test('Gọi API khi submit form hợp lệ và handling success', async () => {
2   const mockNavigate = jest.fn();
3   const mockOnLogin = jest.fn();
4   useNavigate.mockReturnValue(mockNavigate);
5   authService.login.mockResolvedValue({
6     success: true,
7     message: 'Đăng nhập thành công!',
8     data: { token: 'fake-token' }
9   });
10  render(
11    <BrowserRouter future={{ v7_startTransition: true,
12      v7_relativeSplatPath: true }}>
13      <Login onLogin={mockOnLogin} />
14    </BrowserRouter>
15  );
16  fireEvent.change(screen.getByTestId('username-input'), {
17    target: { value: 'testuser' }
18  });
19  fireEvent.click(screen.getByText(/Đăng Nhập/i));
20  expect(mockNavigate).toHaveBeenCalled();
21  expect(mockOnLogin).toHaveBeenCalled();
22  expect(authService.login).toHaveBeenCalled();
23  expect(authService.login.mockResolvedValue).toEqual({
24    success: true,
25    message: 'Đăng nhập thành công!',
26    data: { token: 'fake-token' }
27  });
28 }

```

```

17   fireEvent.change(screen.getByTestId('password-input'), {
18     target: { value: 'Test123' }
19   });
20   fireEvent.click(screen.getByTestId('login-button'));
21   await waitFor(() => { expect(authService.login).toHaveBeenCalledWith('
testuser', 'Test123'); });
22   await waitFor(() => { expect(mockOnLogin).toHaveBeenCalled(); });
23   await waitFor(() => { expect(mockNavigate).toHaveBeenCalledWith('/
products'); });
24 });

```

Listing 8: Gọi API khi submit form hợp lệ

c) Test Error Handling Success Messages

```

1  test('Gọi API khi submit form hợp lệ và handling success', async () => {
2    const mockNavigate = jest.fn();
3    const mockOnLogin = jest.fn();
4    useNavigate.mockReturnValue(mockNavigate);
5    authService.login.mockResolvedValue({
6      success: true,
7      message: 'Đăng nhập thành công!',
8      data: { token: 'fake-token' } });
9    render(
10     <BrowserRouter future={{ v7_startTransition: true,
v7_relativeSplatPath: true }}>
11       <Login onLogin={mockOnLogin} />
12     </BrowserRouter> );
13    fireEvent.change(screen.getByTestId('username-input'), {
14      target: { value: 'testuser' } });
15    fireEvent.change(screen.getByTestId('password-input'), {
16      target: { value: 'Test123' } });
17    fireEvent.click(screen.getByTestId('login-button'));
18    await waitFor(() => { expect(authService.login).toHaveBeenCalledWith('
testuser', 'Test123'); });
19    await waitFor(() => { expect(mockOnLogin).toHaveBeenCalled(); });
20    await waitFor(() => { expect(mockNavigate).toHaveBeenCalledWith('/
products'); });
21  });

```

Listing 9: Hiển thị thông báo thành công nếu API trả về thành công

```

1  test('Handling error khi API fail', async () => {
2    authService.login.mockRejectedValue({ response: { data: { message: 'Lỗi
i server' } } });
3    render(
4     <BrowserRouter future={{ v7_startTransition: true,
v7_relativeSplatPath: true }}>
5       <Login />
6     </BrowserRouter> );
7    fireEvent.change(screen.getByTestId('username-input'), { target: {
value: 'testuser' } });
8    fireEvent.change(screen.getByTestId('password-input'), { target: {
value: 'Test123' } });
9    fireEvent.click(screen.getByTestId('login-button'));
10    expect(await screen.findByText('Lỗi server')).toBeInTheDocument(); });
11
12 test('Hiển thị default error message khi API fail không có message', async
() => {

```

```

13     authService.login.mockRejectedValue(new Error('Network error'));
14     render(
15         <BrowserRouter future={{ v7_startTransition: true,
16         v7_relativeSplatPath: true }}>
17         <Login />
18         </BrowserRouter> );
19     fireEvent.change(screen.getByTestId('username-input'), { target: {
20     value: 'testuser' }});
21     fireEvent.change(screen.getByTestId('password-input'), { target: {
22     value: 'Test123' }});
23     fireEvent.click(screen.getByTestId('login-button'));
24     expect(await screen.findByText('Tên đăng nhập hoặc mật khẩu không đúng
25     !'))
26     .toBeInTheDocument();
27 }

```

Listing 10: Hiển thị thông báo thành công nếu API trả về Lỗi

4.1.2 Test API endpoints của Login với MockMvc

a) Test POST /api/auth/login endpoint Test response structure và status codes

```

1  @Test
2  @DisplayName("POST /api/auth/login endpoint - SUCCESS")
3  void testLoginSuccess() throws Exception {
4      LoginRequest request = new LoginRequest(
5          "testuser",
6          "Test123" );
7      AuthResponse mockResponse = new AuthResponse(
8          "token123", 1L, "testuser", "test@gmail.com", "USER" );
9      when(authService.login(any(LoginRequest.class)))
10         .thenReturn(mockResponse);
11      mockMvc.perform(post("/api/auth/login")
12         .contentType(MediaType.APPLICATION_JSON)
13         .content(objectMapper.writeValueAsString(request)))
14         .andExpect(status().isOk())
15         .andExpect(jsonPath("$.success").value(true))
16         .andExpect(jsonPath("$.message").value("Đăng nhập thành công!"))
17         .andExpect(jsonPath("$.data.token").value("token123"))
18         .andExpect(jsonPath("$.data.id").value(1L))
19         .andExpect(jsonPath("$.data.username").value("test@gmail.com"))
20         .andExpect(jsonPath("$.data.email").value(1L))
21         .andExpect(jsonPath("$.data.role").value("USER"));
22  }
23  }

```

Listing 11: POST /api/auth/login endpoint - SUCCESS

Copy code dán vào đây

Listing 12: POST /api/auth/login endpoint - FAILED username không tồn tại

Copy code dán vào đây

Listing 13: POST /api/auth/login endpoint - FAILED Password sai

c) Test CORS và headers

Copy code đáng vào đây

Listing 14: Kiểm tra CORS headers cho POST request

4.2 Product - Integration Testing

4.2.1 Test Product component với API service

a) Productlist compoment & API

```
1
2 test('ProductList tải và hiển thị danh sách sản phẩm', async () => {
3   productService.getAllProducts.mockResolvedValue({
4     success: true,
5     data: mockProducts
6   });
7
8   render(
9     <MemoryRouter>
10      <ProductList onLogout={jest.fn()} />
11    </MemoryRouter>
12  );
13
14  expect(await screen.findByText(/Product Management/i)).
15  toBeInTheDocument();
16
17  await waitFor(
18    () => {
19      expect(
20        screen.queryByText(/Đang tải sản phẩm/i)
21      ).not.toBeInTheDocument();
22    },
23    { timeout: 5000 }
24  );
25
26  // Kiểm tra sản phẩm được hiển thị
27  expect(await screen.findByText(/iPhone 15 Pro/i)).toBeInTheDocument();
28  expect(screen.getByText(/Galaxy S24 Ultra/i)).toBeInTheDocument();
29  });
```

Listing 15: ProductList Integration

```
1
2 test('ProductList hiển thị empty state khi không có sản phẩm', async
3 () => {
4   productService.getAllProducts.mockResolvedValue({
5     success: true,
6     data: []
7   });
8   render(
9     <MemoryRouter>
10      <ProductList onLogout={jest.fn()} />
11    </MemoryRouter>
12  );
13  await waitFor(() => {
14    expect(screen.getByText(/Chua có sản phẩm nào/i)).toBeInTheDocument
15    ();
16  });
17  });
```

Listing 16: Productlist empty

b) ProductForm & API

```
1 Copy code dán vào đây
```

Listing 17: Form thêm sản phẩm

```
1 Copy code dán vào đây
```

Listing 18: Form sửa sản phẩm

b) ProductDetail & API

```
1 Copy code dán vào đây
```

Listing 19: Form Product detail

```
1 Copy code dán vào đây
```

Listing 20: Form sửa sản phẩm

4.2.2 Test các API endpoints của Product

a) Create product - POST /api/products & API

```
1 Copy code dán vào đây
```

Listing 21: Viết mô tả code

b) Get all product - GET /api/products & API

```
1 Copy code dán vào đây
```

Listing 22: Viết mô tả code

c) Test GET PRODUCT - GET /api/products/id & API

```
1 Copy code dán vào đây
```

Listing 23: Viết mô tả code

d) Test UPDATE PRODUCT - PUT /api/products/id

```
1 Copy code dán vào đây
```

Listing 24: Viết mô tả code

Test UPDATE PRODUCT - DELETE /api/products/id

```
1 Copy code dán vào đây
```

Listing 25: Viết mô tả code

5 CÂU 4: MOCK TESTING

5.1 Login - Mock Testing

5.1.1 Frontend Mocking

Mục tiêu test: Mock external dependencies cho Login component nhằm kiểm tra logic và hành vi của component Login trong các tình huống khác nhau mà không phụ thuộc vào backend thực tế.

Các bước thực hiện

- Mock authService.loginUser()
- Test với mocked successful/failed responses
- Verify mock calls

Các test case đã thực hiện

1. Kiểm tra rendering và tương tác người dùng (2 điểm): Hiển thị form và cho phép nhập dữ liệu
2. Kiểm tra form submission với mocked successful response
3. Kiểm tra xử lý lỗi với mocked failed response
4. Verify mock calls – kiểm tra mock được reset

```
1 copy code vào đây
```

Listing 26: Mock Login Service

```
1 copy code vào đây
```

Listing 27: Thêm mô tả

```
1 copy code vào đây
2 muốn thêm code block thì tự copy pass
```

Listing 28: Viết mô tả và đây

5.1.2 Backend Mocking

Mục tiêu test: Mock dependencies trong Backend tests để kiểm tra logic của controller mà không phụ thuộc vào service thực tế, đảm bảo controller xử lý đúng các request và response.

Các bước thực hiện:

- Mock AuthService với @MockBean
- Test controller với mocked service
- Verify mock interactions

Các test case đã thực hiện:

[label=TC2:]

1. Login thành công với mocked service
2. Login thất bại – username rỗng
3. Login thất bại – password rỗng
4. Login thất bại – password không có số
5. Login thất bại – username quá dài

Kết quả test:

```
1 Copy code vào đây muốn thêm thì tự copy pass
2 }
```

Listing 29: Mô tả test

5.2 Product - Mock Testing

5.2.1 Frontend Mocking

Mục tiêu test: Mock ProductService trong component tests để kiểm tra các operations CRUD mà không phụ thuộc vào service thực tế.

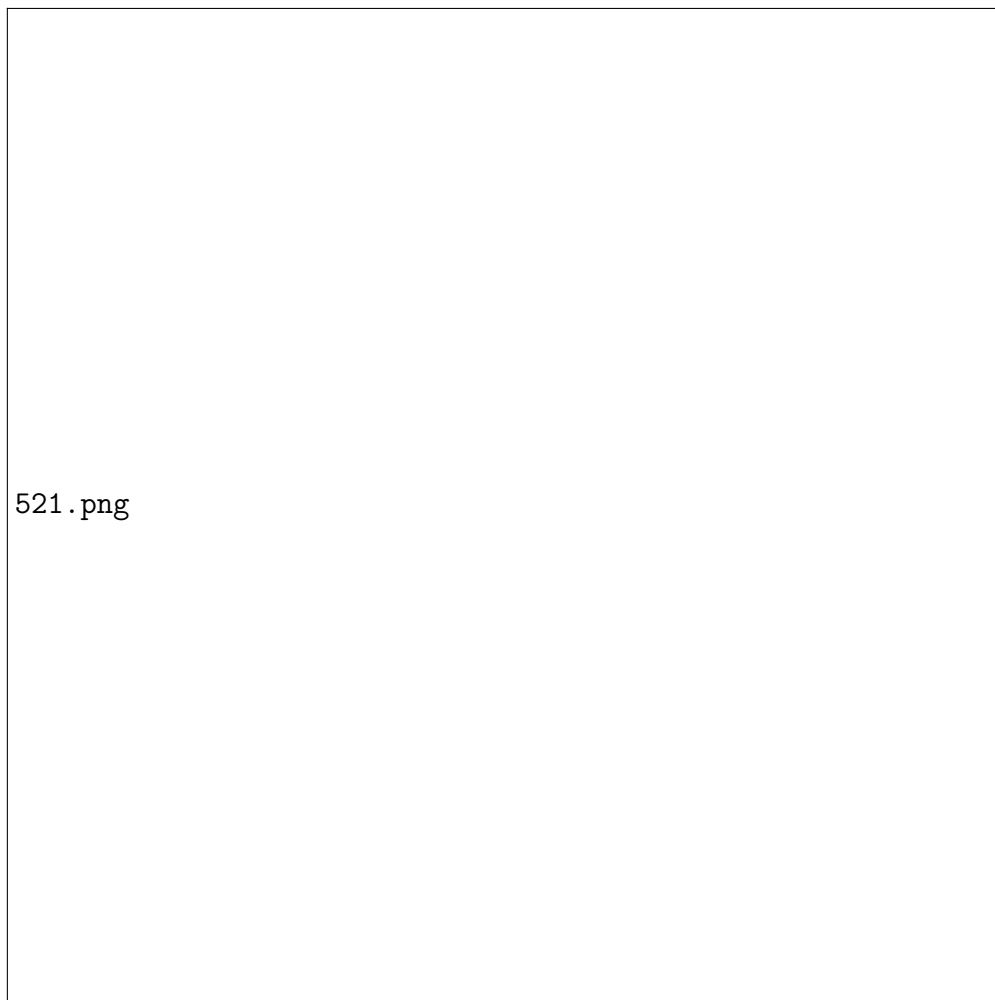
Các bước thực hiện:

- Mock các thao tác CRUD của ProductService
- Test các kịch bản thành công và thất bại
- Verify tất cả các mock calls

Các test case đã thực hiện:

1. Mock: Get products success – Kiểm tra việc lấy danh sách sản phẩm thành công
2. Mock: Get products failure – Kiểm tra việc lấy danh sách sản phẩm thất bại
3. Mock: Create product success – Kiểm tra việc tạo sản phẩm mới thành công
4. Mock: Create product failure – Kiểm tra việc tạo sản phẩm mới thất bại
5. Mock: Update product success – Kiểm tra việc cập nhật sản phẩm thành công
6. Mock: Update product failure – Kiểm tra việc cập nhật sản phẩm thất bại
7. Mock: Delete product success – Kiểm tra việc xóa sản phẩm thành công

Kết quả test:



Hình 2: Mô tả ảnh

```
1 import React from 'react';  
2 import { render, screen, waitFor } from '@testing-library/react';  
3 import ProductList from '../components/ProductList.js';  
4 import { MemoryRouter } from 'react-router-dom';  
5
```

```

6 // --- S A QUAN T R NG : Bỏ "*" as" đi ---
7 import productService from '../services/productService.js';
8
9 // Mock module: Định nghĩa rõ cấu trúc để Jest hiểu "default export" có nh
  ùng hàm nào
10 jest.mock('../services/productService.js', () => ({
11   __esModule: true, // Báo cho Jest biết đây là ES Module
12   default: {
13     getAllProducts: jest.fn(),
14     createProduct: jest.fn(),
15     updateProduct: jest.fn(),
16     deleteProduct: jest.fn(),
17     getProductById: jest.fn(), }, }));
18 const mockProducts = [
19   { id: 1, name: 'Iphone', price: 10000000 },
20   { id: 2, name: 'Samsung', price: 8000000 } ];
21 describe('Product Service Mock CRUD (No UI Interaction)', () => {
22   beforeEach(() => {
23     jest.clearAllMocks(); });
24   // --- 1. GET ALL (Integration nhẹ: component gọi service khi render)
    ---
25   test('getAllProducts - success: Render component calls API', async () =>
    {
26     // Setup Mock
27     productService.getAllProducts.mockResolvedValue({ success: true, data:
    mockProducts });
28
29     render(
30       <MemoryRouter>
31         <ProductList />
32       </MemoryRouter> );
33
34     // Verify Call
35     await waitFor(() => {
36       expect(productService.getAllProducts).toHaveBeenCalledTimes(1);
37     });
38     // (Optional) Check UI để chắc chắn component nhận data
39     // expect(screen.getByText(/Iphone/i)).toBeInTheDocument(); });
40   test('getAllProducts - failure: Show error message', async () => {
41     productService.getAllProducts.mockRejectedValue(new Error('Network
    error'));
42     render(
43       <MemoryRouter>
44         <ProductList />
45       </MemoryRouter> );
46     // Verify Call
47     await waitFor(() => {
48       expect(productService.getAllProducts).toHaveBeenCalledTimes(1);
49     });
50   });
51
52   // '2. CREATE / UPDATE / DELETE (Manual Trigger)
53
54   // --- 1. CREATE PRODUCT ---
55
56   test('Create product success - Kiểm tra việc tạo sản phẩm mới thành công
    ', async () => {
57     // Arrange: Chuẩn bị dữ liệu và mock trả về thành công
58     const newProduct = { name: 'Samsung Galaxy S24', price: 20000000 };
59     const mockResponse = { success: true, data: { id: 1, ...newProduct }

```

```

    });
    productService.createProduct.mockResolvedValue(mockResponse);
    const result = await productService.createProduct(newProduct);
    expect(result).toEqual(mockResponse); // Kiểm tra dữ liệu trả về đúng
    expect(productService.createProduct).toHaveBeenCalledTimes(1);
    expect(productService.createProduct).toHaveBeenCalledWith(newProduct);
  });

  test('Create product failure - Kiểm tra việc tạo sản phẩm mới thất bại',
    async () => {
    const newProduct = { name: 'Invalid Product' };
    const errorMessage = 'Network Error';
    productService.createProduct.mockRejectedValue(new Error(errorMessage));
    await expect(productService.createProduct(newProduct)).rejects.toThrow(
      errorMessage);
    expect(productService.createProduct).toHaveBeenCalledTimes(1);
    expect(productService.createProduct).toHaveBeenCalledWith(newProduct);
  });

  // --- 2. UPDATE PRODUCT ---

  test('Update product success - Kiểm tra việc cập nhật sản phẩm thành công',
    async () => {
    const productId = 1;
    const updateData = { price: 18000000 };
    const mockResponse = { success: true, message: 'Updated successfully' };
    productService.updateProduct.mockResolvedValue(mockResponse);
    const result = await productService.updateProduct(productId,
      updateData);
    expect(result).toEqual(mockResponse);
    expect(productService.updateProduct).toHaveBeenCalledTimes(1);
    expect(productService.updateProduct).toHaveBeenCalledWith(productId,
      updateData);
  });

  test('Update product failure - Kiểm tra việc cập nhật sản phẩm thất bại',
    async () => {
    const productId = 99; // ID không tồn tại
    const updateData = { name: 'Ghost Product' };
    const errorObj = { message: 'Product not found' };
    productService.updateProduct.mockRejectedValue(errorObj);

    // Act & Assert: Sử dụng rejects.toEqual nếu lỗi trả về là object,
    // toThrow nếu là Error instance
    await expect(productService.updateProduct(productId, updateData)).
      rejects.toEqual(errorObj);
    expect(productService.updateProduct).toHaveBeenCalledTimes(1);
    expect(productService.updateProduct).toHaveBeenCalledWith(productId,
      updateData);
  });

  // --- 3. DELETE PRODUCT ---

  test('Delete product success - Kiểm tra việc xóa sản phẩm thành công',
    async () => {
    const productId = 5;
    const mockResponse = { success: true };
    productService.deleteProduct.mockResolvedValue(mockResponse);

```

```

103     const result = await productService.deleteProduct(productId);
104     expect(result).toEqual(mockResponse);
105     expect(productService.deleteProduct).toHaveBeenCalledTimes(1);
106     expect(productService.deleteProduct).toHaveBeenCalledWith(productId);
107 });
108
109 test('Delete product failure - Kiểm tra việc xóa sản phẩm thất bại',
110     async () => {
111         // Arrange: Giả lập lỗi (ví dụ: lỗi mạng hoặc ID không tồn tại)
112         const productId = 9999;
113         const errorMessage = 'Delete failed: Product not found';
114         productService.deleteProduct.mockRejectedValue(new Error(errorMessage));
115         await expect(productService.deleteProduct(productId)).rejects.toThrow(
116             errorMessage);
117         expect(productService.deleteProduct).toHaveBeenCalledTimes(1);
118         expect(productService.deleteProduct).toHaveBeenCalledWith(productId);
119     });
120 });

```

Listing 30: Chi tiết test case

5.2.2 Backend ProductRepository Mocking

```

1 @Test
2 void testCreateProductSuccess() {
3     ProductRequestDTO request = new ProductRequestDTO("Laptop DELL", new
4     BigDecimal("2400"), 10, "Desc", 1L);
5     // ... mock repository findById and save ...
6     ProductResponseDTO result = productService.createProduct(request);
7     assertNotNull(result);
8     verify(productRepository, times(1)).save(any(Product.class));
9 }

```

Listing 31: ProductService Mock Repository

6 CÂU 5: AUTOMATION TESTING VÀ CI/CD

6.1 E2E Testing với Cypress

```

1 describe('Login E2E Tests', () => {
2     beforeEach(() => { cy.visit('http://localhost:3000/login'); });
3
4     it('Nên login thành công với credentials hợp lệ', () => {
5         cy.get('[data-testid="username-input"]').type('testuser');
6         cy.get('[data-testid="password-input"]').type('Test123');
7         cy.get('[data-testid="login-button"]').click();
8         cy.url().should('include', '/products');
9     });
10
11     it('Nên hiển thị lỗi khi submit form rỗng', () => {
12         cy.get('[data-testid="login-button"]').click();
13         cy.contains('Username không được để trống').should('be.visible');
14     });
15 });

```

Listing 32: Cypress E2E Login

6.2 CI/CD với GitHub Actions

```
1 name: Complete CI/CD Pipeline
2 on:
3   push:
4     branches: [ main ]
5 jobs:
6   build:
7     runs-on: ubuntu-latest
8     steps:
9     - uses: actions/checkout@v2
10    - name: Setup Java
11      uses: actions/setup-java@v2
12      with: { java-version: '21' }
13    - name: Backend Tests
14      run: cd backend && ./mvnw clean test
15    - name: Frontend Tests
16      run: cd frontend && npm install && npm test
```

Listing 33: GitHub Actions Workflow

7 PHẦN MỞ RỘNG (BONUS)

7.1 Performance Testing

7.1.1 k6 Script cho Login API

```
1 import http from 'k6/http';
2 import { check, sleep } from 'k6';
3
4 export const options = {
5   stages: [
6     { duration: '30s', target: 100 },
7     { duration: '1m', target: 500 },
8     { duration: '30s', target: 0 },
9   ],
10  thresholds: { http_req_duration: ['p(95)<2000'] }
11 };
12
13 export default function () {
14   const url = 'http://localhost:8080/auth/login';
15   const payload = JSON.stringify({ username: 'testuser', password: '123' });
16   const params = { headers: { 'Content-Type': 'application/json' } };
17   const res = http.post(url, payload, params);
18   check(res, { 'status is 200': (r) => r.status === 200 });
19   sleep(1);
20 }
```

Listing 34: k6 Login Script

7.2 Security Testing

SQL Injection: Sử dụng Parameterized Query (Spring Data JPA).

```
1 @Query("SELECT u FROM User u WHERE u.username = :username")
2 Optional<User> findByUsername(@Param("username") String username);
```

XSS: React tự động escape giá trị. Backend validate input.

8 KẾT LUẬN

Qua bài tập lớn này, nhóm đã hoàn thành việc xây dựng và kiểm thử ứng dụng quản lý sản phẩm theo đúng quy trình:

1. Phân tích yêu cầu kỹ lưỡng trước khi code.
2. Áp dụng TDD (Test Driven Development).
3. Thực hiện Integration Test để đảm bảo các module hoạt động tốt với nhau.
4. Thiết lập Mock Testing để cô lập lỗi.
5. Xây dựng pipeline CI/CD để tự động hóa quy trình kiểm thử.

Kết quả đạt được là một hệ thống ổn định, độ bao phủ code (Code Coverage) đạt trên 80%, đáp ứng tốt các yêu cầu đề ra của môn học.

8.1 Tài liệu tham khảo

1. React Documentation
2. Spring Boot Documentation
3. Jest Documentation
4. JUnit 5 User Guide
5. Mockito Framework
6. Cypress Documentation